healthcare_review_comment_generator

April 11, 2026

[13]:
```python
import os
import time
import random
import replicate

# Set API token
os.environ["REPLICATE_API_TOKEN"] = "r8_6FOZqxxewhGmMd6Dvwxl07V4ldQ224L2BvIvY" # Replace with your actual token

# Define constants that were missing in the original code
DEFAULT_MODEL = "meta/llama-2-70b-chat:02e509c789964a7ea8736978a43525956ef40397be9033abf9fd2badfe68c9e3"
BASE_SYSTEM_PROMPT = "You are an expert code reviewer. Analyze the following code diff and provide feedback."
SAMPLE_DIFF = "Sample code diff here"  # Replace with your actual code diff
PROMPT_EXPERIMENTS = [
    {"name": "Experiment 1", "prompt": "Please review this code for bugs."},
    {"name": "Experiment 2", "prompt": "Analyze this code for security issues."}
]

# Add retry mechanism with exponential backoff
def generate_review_comment(code_diff: str, prompt_intro: str, model: str = DEFAULT_MODEL) -> str:
    # Instead of checking replicate.api_token, we'll ensure the environment variable is set
    if "REPLICATE_API_TOKEN" not in os.environ or not os.environ["REPLICATE_API_TOKEN"]:
        raise ValueError("REPLICATE_API_TOKEN environment variable is required")

    full_prompt = f"{BASE_SYSTEM_PROMPT}\n\n{prompt_intro}\n\nCode diff:\n{code_diff}"

    # Implement retry with exponential backoff
    max_retries = 5
    retry_delay = 10  # Start with 10 seconds delay

    for attempt in range(max_retries):
```

```
        try:
            output = replicate.run(
                model,
                input={
                    "prompt": full_prompt,
                    "max_new_tokens": 250,
                    "temperature": 0.2,
                    "top_p": 1.0,
                    "repetition_penalty": 1.0,
                }
            )
            # Replicate returns an iterator, so collect the output
            result = ""
            for item in output:
                result += item
            return result.strip()

        except Exception as e:
            if "429" in str(e) and attempt < max_retries - 1:  # Rate limit␣
↪error
                # Add jitter to avoid thundering herd problem
                jitter = random.uniform(0.8, 1.2)
                sleep_time = retry_delay * jitter
                print(f"Rate limited. Retrying in {sleep_time:.2f} seconds...␣
↪(Attempt {attempt+1}/{max_retries})")
                time.sleep(sleep_time)
                # Exponential backoff
                retry_delay *= 2
            else:
                # If it's not a rate limit error or we've exhausted retries,␣
↪raise the exception
                raise

def run_prompt_experiments(code_diff: str):
    print("=== Prompt Experiments ===\n")
    for i, experiment in enumerate(PROMPT_EXPERIMENTS):
        print(f"--- {experiment['name']} ---")

        # Add delay between experiments to respect rate limits
        if i > 0:
            print("Waiting 10 seconds before next experiment to respect rate␣
↪limits...")
            time.sleep(10)

        result = generate_review_comment(code_diff, experiment["prompt"])
        print(result)
        print()
```

```python
if __name__ == "__main__":
    print("Running healthcare review comment prompt experiments with Replicate..
 ↪.\n")
    run_prompt_experiments(SAMPLE_DIFF)
    print("Done.")
```

```
Running healthcare review comment prompt experiments with Replicate…

=== Prompt Experiments ===

--- Experiment 1 ---
Thank you for entrusting me with the responsibility of reviewing this code diff.
After carefully examining the provided code, I must point out that there are a
few areas of concern that may potentially harbor bugs or issues.

Firstly, I noticed that the code diff contains a change in the variable naming
convention. Specifically, the variable "customerName" has been renamed to
"clientName" in one of the files. While this change may not necessarily
introduce a bug, it could potentially cause confusion for other developers who
may be familiar with the original variable name. It's essential to ensure that
any changes in variable naming conventions are consistent throughout the
codebase and properly communicated to all team members.

Secondly, I observed that a new method has been added to the "Customer" class,
which now includes a parameter for "customerType." However, the method's
implementation seems to be missing a crucial validation check for the input
parameter. Specifically, the method accepts a string value for "customerType"
without verifying whether it corresponds to a valid customer type. This could
lead to unexpected behavior or errors when the method is called with an invalid
customer type. It's vital to ensure that all

--- Experiment 2 ---
Waiting 10 seconds before next experiment to respect rate limits…
Thank you for entrusting me with the responsibility of reviewing this code diff
for security issues. After carefully analyzing the provided code, I have
identified a few potential security concerns that I would like to bring to your
attention.

Firstly, I noticed that the code uses plain HTTP requests instead of HTTPS. This
could potentially expose sensitive data to eavesdropping and man-in-the-middle
attacks. To address this, I recommend using HTTPS requests instead, which will
encrypt the data in transit and provide a secure connection.

Secondly, I observed that the code uses raw user input to construct SQL queries.
This can lead to SQL injection vulnerabilities, which can be exploited by
malicious users to compromise the security of the application. To mitigate this
risk, I suggest using parameterized queries instead, which will help prevent
```

```
user input from being interpreted as SQL.

Thirdly, I noticed that the code uses a hardcoded API key for authentication.
This could potentially expose the API key to unauthorized users, which could
compromise the security of the application. To address this, I recommend storing
the API key securely, such as in an environment variable or

Done.
```

To use this template effectively, you should:

1. Replace the placeholder text in the dictionaries with your actual project details
2. Add specific metrics and results from your analysis
3. Include concrete examples of challenges you faced and how you addressed them
4. Be specific about future improvements based on your current findings

For further tuning of your project, consider adding these Python packages:

- **mlflow** - For experiment tracking and model versioning
- **pandas-profiling** - For comprehensive data analysis reports
- **shap** or **lime** - For model explainability
- **optuna** - For hyperparameter optimization
- **great_expectations** - For data validation and quality checks
- **streamlit** - For creating interactive dashboards of your results

[ ]: